Diagrama de Clases UML — Plataforma de gestión y predicción académica

Proyecto semestral · Desarrollo de Software Orientado a Objetos · Java

Leyenda

Módulo 1 · Núcleo académico (Integrante 1)

Módulo 2 · Estudiante, metas y horario (Integrante 2)

Módulo 3 · Análisis, interfaz y ejecución (Integrante 3)

- privado # protegido + público subrayado = static *cursiva* = abstracto

◆ composición ◇ agregación → asociación ▷ herencia - - ▷ implementa - - → «usa»

Main

+ main(args: String[]): void
- cargarDatosDePrueba(): Estudiante

«usa»

InterfazConsola

- estudiante: Estudiante
- gestorAlertas: GestorAlertas
- gestorRecomendaciones: GestorRecomendaciones
- scanner: Scanner

+ InterfazConsola(estudiante: Estudiante)
+ iniciar(): void
- mostrarMenu(): void
- registrarCurso(): void
- registrarEvaluacion(): void
- registrarNota(): void
- registrarMeta(): void
- mostrarMetas(): void
- mostrarRecomendaciones(): void
- mostrarHorario(): void
- mostrarAlertas(): void
- leerNota(): double
- leerEntero(mensaje: String): int

Estudiante

- codigo: String
- nombres: String
- carrera: String
- cursos: ArrayList<Curso>
- metas: ArrayList<Meta>
- horario: HorarioEstudio

+ Estudiante(codigo: String, nombres: String)
+ Estudiante(codigo: String, nombres: String, carrera: String)
+ agregarCurso(curso: Curso): boolean
+ buscarCurso(codigo: String): Curso
+ agregarMeta(meta: Meta): void
+ calcularTotalCreditos(): int
+ calcularPromedioPonderadoActual(): double
+ getCodigo(): String
+ getNombres(): String
+ getCarrera(): String
+ getCursos(): ArrayList<Curso>
+ getMetas(): ArrayList<Meta>
+ getHorario(): HorarioEstudio

GestorAlertas

- alertas: ArrayList<Alerta>

+ GestorAlertas()
+ generarAlertas(cursos: ArrayList<Curso>): ArrayList<Alerta>
+ contarAlertasPorNivel(nivel: String): int
+ getAlertas(): ArrayList<Alerta>

GestorRecomendaciones

- recomendaciones: ArrayList<String>

+ GestorRecomendaciones()
+ generarRecomendaciones(estudiante: Estudiante): ArrayList<String>
- recomendarPorCurso(curso: Curso): void
- recomendarPorMetas(metas: ArrayList<Meta>): void
+ getRecomendaciones(): ArrayList<String>

«abstract»
Meta

nombre: String
notaObjetivo: double

+ Meta(nombre: String, notaObjetivo: double)
+ *calcularNotaMinimaRequerida(): double*
+ *getTipo(): String*
+ esAlcanzable(): boolean
+ obtenerEstado(): String
+ getNombre(): String
+ getNotaObjetivo(): double

HorarioEstudio

- horasDisponibles: HashMap<String, Integer>
- bloques: ArrayList<BloqueEstudio>
+ DIAS: String[] {final}

+ HorarioEstudio()
+ definirDisponibilidad(dia: String, horas: int): boolean
+ generarHorario(cursos: ArrayList<Curso>): void
+ obtenerBloquesPorDia(dia: String): ArrayList<BloqueEstudio>
+ calcularTotalHoras(): int
- calcularHorasRecomendadas(curso: Curso): int
+ getBloques(): ArrayList<BloqueEstudio>

«interface»
Alertable

+ estaEnRiesgo(): boolean
+ obtenerNivelRiesgo(): String

Alerta

- curso: Curso
- nivel: String
- mensaje: String

+ Alerta(curso: Curso, nivel: String, mensaje: String)
+ getCurso(): Curso
+ getNivel(): String
+ getMensaje(): String
+ toString(): String

«interface»
Recomendable

+ generarRecomendacion(notaNecesaria: double): String

MetaPromedio

- categoria: String
- cursos: ArrayList<Curso>

+ MetaPromedio(categoria: String, notaObjetivo: double, cursos: ArrayList<Curso>)
+ calcularNotaMinimaRequerida(): double
+ getTipo(): String
+ getCategoria(): String

MetaAprobacion

- curso: Curso

+ MetaAprobacion(curso: Curso)
+ MetaAprobacion(curso: Curso, notaObjetivo: double)
+ calcularNotaMinimaRequerida(): double
+ getTipo(): String
+ getCurso(): Curso

BloqueEstudio

- dia: String
- horas: int
- curso: Curso

+ BloqueEstudio(dia: String, horas: int, curso: Curso)
+ getDia(): String
+ getHoras(): int
+ getCurso(): Curso
+ toString(): String

Curso

- codigo: String
- nombre: String
- creditos: int
- evaluaciones: ArrayList<Evaluacion>
+ CREDITOS_POR_DEFECTO: int = 3 {final}

+ Curso(codigo: String, nombre: String)
+ Curso(codigo: String, nombre: String, creditos: int)
+ agregarEvaluacion(evaluacion: Evaluacion): boolean
+ buscarEvaluacion(codigo: String): Evaluacion
+ registrarNota(codigoEvaluacion: String, nota: double): boolean
+ calcularSumaPesos(): double
+ pesosCompletos(): boolean
+ calcularPuntajeAcumulado(): double
+ calcularPesoPendiente(): double
+ calcularPromedioActual(): double
+ calcularNotaNecesaria(notaObjetivo: double): double
+ obtenerEvaluacionesPendientes(): ArrayList<Evaluacion>
+ estaEnRiesgo(): boolean
+ obtenerNivelRiesgo(): String
+ getCodigo(): String
+ getNombre(): String
+ getCreditos(): int
+ getEvaluaciones(): ArrayList<Evaluacion>

«abstract»
Evaluacion

codigo: String
nombre: String
peso: double
calificacion: Calificacion

+ Evaluacion(codigo: String, nombre: String, peso: double)
+ registrarNota(nota: double): boolean
+ registrarNota(nota: double, observacion: String): boolean
+ estaRendida(): boolean
+ getNota(): double
+ calcularAporte(): double
+ *getTipo(): String*
+ *generarRecomendacion(notaNecesaria: double): String*
+ getCodigo(): String
+ getNombre(): String
+ getPeso(): double
+ getCalificacion(): Calificacion

PracticaCalificada

- numeroPractica: int

+ PracticaCalificada(codigo: String, nombre: String, peso: double)
+ PracticaCalificada(codigo: String, nombre: String, peso: double, numeroPractica: int)
+ getTipo(): String
+ generarRecomendacion(notaNecesaria: double): String
+ getNumeroPractica(): int

Trabajo

- esGrupal: boolean

+ Trabajo(codigo: String, nombre: String, peso: double)
+ Trabajo(codigo: String, nombre: String, peso: double, esGrupal: boolean)
+ getTipo(): String
+ generarRecomendacion(notaNecesaria: double): String
+ isEsGrupal(): boolean

ExamenParcial

- unidad: int

+ ExamenParcial(codigo: String, nombre: String, peso: double)
+ ExamenParcial(codigo: String, nombre: String, peso: double, unidad: int)
+ getTipo(): String
+ generarRecomendacion(notaNecesaria: double): String
+ getUnidad(): int

Calificacion

- nota: double
- observacion: String
+ NOTA_MINIMA: double = 0 {final}
+ NOTA_MAXIMA: double = 20 {final}
+ NOTA_APROBATORIA: double = 10.5 {final}

+ Calificacion(nota: double)
+ Calificacion(nota: double, observacion: String)
+ esNotaValida(nota: double): boolean
+ estaAprobada(): boolean
+ getNota(): double
+ getObservacion(): String